

Library Management System

- Course: Object Oriented Programming Lab
- Student: Diyali Das
- Roll No: 62 | Section: A
- Department: IT
- Session: Spring 2026

Problem Statement (1)

- Manual & semi- digital systems used
- Physical registers/spreadsheets
- Hard to scale with growing users
- Error- prone and inefficient

Problem Statement (2)

- No real- time book tracking
- Manual search required
- Delays and incorrect information
- User inconvenience

Problem Statement (3)

- Manual due date tracking
- No reminders
- Manual fine calculation
- Possible inconsistencies

Problem Statement (4)

- Difficult record maintenance
- Hard to retrieve past data
- No proper analytics
- Risk of data loss

Solution Overview

- Automated Library Management System
- Built using Java
- Uses OOP principles
- Structured data management

System Features

- Real- time book tracking
- User management
- Borrow/return automation
- Fine calculation
- Record retrieval

System Architecture

- Presentation Layer (Console UI)
- Business Logic Layer
- Data Handling Layer
- Layered architecture

Modules

- Book Management
- User Management
- Transaction Management
- Fine Calculation

OOP - Encapsulation

- Classes: Book, User, Transaction
- Private data members
- Public getter/setter methods
- Ensures data security

OOP - Abstraction

- Complex operations hidden
- Simple method interfaces
- Borrow process simplified
- Improves usability

OOP - Inheritance

- Base class: User
- Derived: Admin, Student
- Code reuse
- Better organization

OOP - Polymorphism

- Method overloading
- Method overriding
- Flexible implementation
- Handles multiple scenarios

Implementation Details (1)

- Language: Java
- Classes created for entities
- Book, User, Transaction classes
- Object- based design

Implementation Details (2)

- Book stores ID, title, status
- User stores user data
- Transaction stores issue/return dates
- Structured representation

Borrow & Return Logic

- Check availability before borrow
- Update status after issuing
- Store issue and due date
- Return updates records

Fine Calculation

- Based on delay in days
- Automated calculation
- Consistent and fair
- Improves accountability

Data Structures Used

- ArrayLists
- Dynamic data storage
- Efficient retrieval
- No database required initially

Testing Approach

- Unit testing
- Integration testing
- System testing
- End- to- end validation

Test Cases

- Borrow available book
- Borrow unavailable book
- Return on time
- Return after due date

Error Handling

- Input validation
- Exception handling (try- catch)
- Logical error checks
- User- friendly messages

Git Workflow

- Version control using Git
- Repository initialization
- Branching for features
- Merging after testing

Conclusion

- Automates library operations
- Reduces errors
- Improves efficiency
- Applies OOP concepts

Future Scope (1)

- GUI using Java Swing/JavaFX
- Better user interface
- Improved usability

Future Scope (2)

- Database integration
- MySQL/SQLite
- Permanent data storage

Future Scope (3)

- Web- based system
- Spring Boot framework
- Access from multiple devices

Future Scope (4)

- User authentication
- Online reservation
- Notifications
- Barcode/QR system

UML Design

- Main uses Library
- Library has Books
- Book: ID, Title, Borrowed status